ZYVORAI LABS · CUSTOMER DOCUMENTATION

# Zyvor Argus

Page-by-Page Product Manual

2026-09-05

# Page-by-page guides

Each guide follows: Purpose → When to use it → How to get there → What you can do → Related pages.

Every route is also listed in the complete page index.

### Api

| Page | What it covers |
|---|---|
| API contract diff | Static OpenAPI breaking-change diff between two specs (URL, git::, or inline JSON) — no live target, no engagement gate. |
| API contract | Validate REST endpoints against an OpenAPI schema (Forge preset available). |
| Auth & session | Login → reusable session file → logout / expiry / negative auth checks. |
| Contract verify | HAR-derived consumer contract verification against a live provider (status, content-type, top-level JSON keys) — requires an active_recon engagement. |
| Live data | Assert WebSocket / SSE streams and optional live-region updates. |

### Console

| Page | What it covers |
|---|---|
| Ask Zyra | Citation-first product knowledge Q&A — optional Qdrant hybrid RAG. Read-only; does not mutate cluster state. Aligned with Zyra copilot branding across the Zyvor platform. |

### Journeys

| Page | What it covers |
|---|---|
| AI test | Describe a goal; the autonomous browser agent drives the app toward it. |
| Flow test | Multi-step user journey (English or step DSL) with optional login/session, video, and Playwright `trace.zip`. |
| HAR record / replay | Record a HAR while browsing routes, or replay the UI against a captured HAR (offline / deterministic network). |
| Import codegen | Paste Playwright codegen JS/TS and convert it into Zyvor Argus flow steps (optionally run immediately). |

### Operations

| Page | What it covers |
|---|---|
| Findings | Collected issues from API, auth, live-data, vitals, and audit jobs with export/clear. |
| QA Runs | History table of QA runs with pass/fail chips and sparkline trends. |
| Schedules | Turn smoke, audit, ping, TLS, flow, route sweep, API, realtime, or vitals into a 5 min–6 h loop. |
| Test health | Worst-offender ranking by fail count, fail %, and flaky badge from the per-test index. |
| Videos | Browse and download recorded journey videos / traces from recent jobs. |

## Overview

| Page | What it covers |
|---|---|
| Command palette | ⌘K / Ctrl-K or header Search — spotlight to jump to any action, panel, or Ask Zyra. |
| Hero status | Cluster/app health banner with pods, replicas, last QA run, pass rate, next schedule, and knowledge (Ask Zyra) stat tile. |
| Live job panel | Live panel for the running job — macOS Terminal chrome, syntax-colored streaming log, per-test chips, Stop / download. |
| Pods | Pod cards with phase, restarts, and click-through logs; optional cluster events toggle. |
| Workloads | Deployment and CronJob strip for the argus namespace (when kube access is available). |
| Mission Control | Live Mission Control console — grouped side rail (Console / Testing / Security / Operations), dark theme, status hero, workloads, pods, category action panels, requirements, schedules, findings, and QA run history. |
| Login | Mission Control sign-in when `DASHBOARD_PASSWORD` is set. Uses the same dark/light design system as the dashboard (charcoal default, blue accent buttons). Argus Enterprise uses Keycloak/OIDC with demo accounts `demo` / `demo` and `ssouser` / `Sso@321` (see the Enterprise SSO chapter in this manual). |

## Pipeline

| Page | What it covers |
|---|---|
| Create from English | Turn an English description into a one-off test (LLM when keyed, heuristic otherwise). |
| Discover coverage | Scan repo code/docs for untested routes and pages, then propose coverage. |
| Generate tests | Generate Playwright tests from a product spec (local path or GitHub). |
| Run tests | Run ▶ Smoke (optional grep/shard) or ▶ Full LangGraph pipeline from local or GitHub specs. |

## Probes

| Page | What it covers |
|---|---|
| Load test | Simple concurrency load test for latency under pressure. |
| Uptime ping | HTTP uptime / latency checks across a list of URLs. |
| Probes | One-shot network & security probes: redirects, headers, cookies, robots, exposed paths, API, sitemap, DNS, CORS, compression. |
| TLS check | Certificate and DNS health for a hostname. |

## Quality

| Page | What it covers |
|---|---|
| Site audit | Crawl pages for a11y (axe), broken links, SEO, console, perf, and security headers. |
| Crawl & test | Crawl an arbitrary site and validate every discovered page. |

| Page | What it covers |
|---|---|
| Flaky check | Re-run the suite N times and surface unstable tests. |
| Web Vitals | Measure Core Web Vitals (LCP / CLS / INP) with optional device and network throttle. |

## Requirements

| Page | What it covers |
|---|---|
| Requirements impact | Impact view for requirements the pipeline ingested: shared data models and |
| Requirements | Versioned requirements the pipeline ingested — source ( `github` , `document` , |

## Security

| Page | What it covers |
|---|---|
| Attack chain | Chains exploit-PoC steps via an LLM planner to confirm a multi-step escalation path — same sandbox and opt-ins as Exploit PoC. |
| Auth attack scan | Auth hygiene: JWT alg=none, cookie flags, login enum hints (no brute force) — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED. |
| Chaos inject | Client-side fault injection (latency / loss / reset / dependency timeout) while a flow or smoke control test observes — needs ZYVOR_CHAOS_INJECTION_ENABLED, exploit engagement, and target consent. |
| Chaos webhook | Trigger a customer-owned chaos experiment webhook, then run a control test with the same resilience rubric and gates as Chaos inject. |
| Cloud pentest | Credentialed AWS/GCP/Azure CLI enumeration of a described finding in the sandbox — same opt-ins and credential-reference rules as Host pentest. |
| CSRF probe | Target CSRF posture: forms without tokens, cookies without SameSite — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED. |
| CVE lookup | Read-only: fingerprints tech/versions and checks them against OSV.dev. No PoC is generated or run — requires a security engagement. |
| DAST scan | Bounded DAST aggregator (headers, injection, CSRF, open-redirect; optional nuclei) — requires an exploit-tier engagement and ZYVOR_DAST_SCAN_ENABLED. |
| DB assert | Read-only SELECT-only assertion (row_count / cell_equals / column_values) against Postgres, MySQL, or SQLite — needs ZYVOR_DB_TESTING_ENABLED and an engagement; DSN is an env-var reference. |
| Exploit PoC | LLM-generated, non-destructive verification of a described finding, executed in a locked-down Kubernetes sandbox — requires an exploit-tier engagement and an execution opt-in. |
| Host pentest | Credentialed SSH enumeration of a described finding via paramiko in the sandbox — needs a third, independent credentialed-pentest opt-in; credentials are always env-var references, never raw values. |
| IDOR scan | Bounded adjacent-numeric-ID comparison for possible IDOR — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED. |
| Injection scan | Systematic SQLi / reflected-XSS / path-traversal probes — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED. |

| Page | What it covers |
| --- | --- |
| LLM red-team | Attacker/judge adversarial-prompt battery against Ask Zyra — prompt injection, system-prompt leak, excessive agency, jailbreak, PII/secret leak — requires a security engagement. |
| Misconfig scan | Tech/version fingerprinting, wordlist-driven path discovery, security-header value grading, and DNS hygiene checks — requires a security engagement. |
| Port scan | Bounded TCP connect scan of common service ports (≤64) — requires an active_recon engagement. |
| SCA scan | Client-side library/license fingerprinting and/or local-checkout pip-audit/npm audit — URL mode needs an engagement; checkout mode is operator-local. |
| Security engagements | Create/revoke the admin-issued, target-scoped authorization required before running misconfig scan, CVE lookup, SCA, DAST / network-attack jobs, LLM red-team, or sandboxed exploit tiers. |
| SSRF probe | Probes URL-like query params for server-side fetch / SSRF signals — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED. |
| TLS cipher scan | TLS protocol and weak-cipher grading beyond basic cert metadata — requires an active_recon engagement. |

### Visual

| Page | What it covers |
| --- | --- |
| Compare URLs | Side-by-side visual diff of two URLs (e.g. staging vs production). |
| Visual regression | Pixel-compare visual snapshots against baselines; optionally update baselines. |
| Route sweep | Screenshot many routes (or auto-crawl) and diff against route-sweep baselines. |
| Screenshot | Capture desktop / tablet / mobile screenshots of any URL. |

---

60 guides. Regenerate: `node scripts/customer-docs/generate-guide-index.mjs`.

---

# API contract diff

### Purpose

Static OpenAPI breaking-change diff between two specs (URL, git::, or inline JSON) — no live target, no engagement gate.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/api-contract-diff`
- UI: Mission Control → **API** panel → **API contract diff** (side rail, or ⌘K / Ctrl-K **Search**)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **API contract diff**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# API contract

### Purpose

Validate REST endpoints against an OpenAPI schema (Forge preset available).

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/api-contract`
- UI: Mission Control → **API** → **API contract** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **API contract**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Auth & session

### Purpose

Login → reusable session file → logout / expiry / negative auth checks.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/auth`
- UI: Mission Control → **API** → **Auth & session** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Auth & session**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Contract verify

### Purpose

HAR-derived consumer contract verification against a live provider (status, content-type, top-level JSON keys) — requires an active_recon engagement.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/contract-verify`
- UI: Mission Control → **API** panel → **Contract verify** (side rail, or ⌘K / Ctrl-K **Search**)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Contract verify**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Live data

### Purpose

Assert WebSocket / SSE streams and optional live-region updates.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/realtime`
- UI: Mission Control → **API** → **Live data** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Live data**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Ask Zyra

### Purpose

Citation-first product knowledge Q&A — optional Qdrant hybrid RAG. Read-only; does not mutate cluster state. Aligned with **Zyra** copilot branding across the Zyvor platform.

### When to use it

- You need grounded answers from ingested manuals, API docs, runbooks, or migration guides
- You want citations and confidence scores before acting on an answer
- Live cluster diagnostics are optional ( `ENABLE_LIVE_CLUSTER_TOOLS` ) — still read-only

### How to get there

- Surface: `/dashboard/ask` · hash `#ask`
- UI: Mission Control → **Console** panel → **Ask Zyra** (side rail, or ⌘K → “Ask Zyra”)

### Operate from the console (UX)

1. Install optional extras: `pip install -e ".[knowledge]"`, start Qdrant, set `LLM_API_KEY` — see Tutorial 14. For Ollama labs without an embeddings API, set `EMBEDDING_BACKEND=fastembed`.
2. Open **Ask Zyra**, pick product / document-type filters, ask a question.
3. Review citations, confidence badge, and ⧉ **copy MD** for Slack/Jira.
4. Header **knowledge lamp** shows ready / degraded / offline.

Without `[knowledge]` extras, Mission Control still works; Ask Zyra shows offline with install hints.

### Related pages

- Tutorial 14 — Ask Zyra
- Using the Dashboard
- Mission Control
- Page index

# AI test

### Purpose

Describe a goal; the autonomous browser agent drives the app toward it.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/ai-test`
- UI: Mission Control → **Journeys** → **AI test** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **AI test**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Flow test

### Purpose

Multi-step user journey (English or step DSL) with optional login/session, video, and Playwright `trace.zip`.

### When to use it

- Prove a critical path after deploy (signup, checkout, create VM, …)
- Capture a single end-to-end video for stakeholders
- Reuse a session from Auth & session

### How to get there

- Surface: `/dashboard/actions/flow`
- UI: Mission Control → **Journeys** → **Flow test** (or ⌘K → "Flow")

### Operate from the console (UX)

1. Enter the app URL and either English ("go to products, click Schedule Demo, assert contact") or one step per line.
2. Optional: login user/password, reuse a saved session file, pick browser / device / throttle.
3. Keep **record video** on, then 🎬 **Run journey**.
4. Watch the live job panel; download video + trace when finished.

#### Step language (excerpt)

`goto`, `click`, `hover`, `fill`, `select`, `upload`, `download`, `dialog`, `iframe`, `drag`, `press`, `clock`, `wait`, `wait until`, `assert`, `assert url`, `assert api`, `assert aria`, `assert not`, `assert count`, `assert value`.

CLI equivalent: `argus flow run <url> --steps file`.

5. **Empty / fail:** Check health, auth, and domain dependencies.
6. **Success:** Live data loads; mutations complete without error toasts.

### Related pages

- HAR record / replay
- Import codegen
- Auth & session
- Getting Started
- Page index

# HAR record / replay

### Purpose

Record a HAR while browsing routes, or replay the UI against a captured HAR (offline / deterministic network).

### When to use it

- Freeze API responses for flaky backends
- Diff UI behavior with vs without live network
- Share a network fixture with the team ( `ZYVOR_HAR_PATH` )

### How to get there

- Surface: `/dashboard/actions/har`
- UI: Mission Control → **Journeys** → **HAR record / replay**

### Operate from the console (UX)

1. **Record** — set mode `record`, enter URL + routes ( `/` , `/pricing` , …). HAR path can auto-generate.
2. **Replay** — set mode `replay`, point at the HAR file, optional expect-text, optional “allow missing routes”.
3. Click **Run** and review the live log.
4. CLI: `argus api har-replay <url> --mode record|replay --har out.har`.
5. **Empty / fail:** Check health, auth, and domain dependencies.
6. **Success:** Live data loads; mutations complete without error toasts.

### Related pages

- Flow test
- Route sweep
- Getting Started
- Page index

# Import codegen

### Purpose

Paste Playwright codegen JS/TS and convert it into Zyvor Argus flow steps (optionally run immediately).

### When to use it

- You already recorded a journey with `playwright codegen`
- You want flow DSL without rewriting clicks by hand

### How to get there

- Surface: `/dashboard/actions/import-codegen`
- UI: Mission Control → **Journeys** → **Import codegen**

### Operate from the console (UX)

1. Paste codegen output ( `page.goto` , `getByRole().click` , `fill` , …).
2. Optionally tick **run as flow** and provide the app URL.
3. Click 📥 **Import** — inspect emitted steps; run if requested.
4. CLI: `argus test import-codegen script.js [--run --url …]` .
5. Local headed record alternative: `npm run record-flow -- <url> out.flow.json` .
6. **Empty / fail:** Check health, auth, and domain dependencies.
7. **Success:** Live data loads; mutations complete without error toasts.

### Related pages

- Flow test
- Getting Started
- Page index

# Findings

### Purpose

Collected issues from API, auth, live-data, vitals, and audit jobs with export/clear.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/findings`
- UI: Mission Control → **Operations** → **Findings** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Findings**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# QA Runs

### Purpose

History table of QA runs with pass/fail chips and sparkline trends.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/runs`
- UI: Mission Control → **Operations** → **QA Runs** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **QA Runs**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Schedules

### Purpose

Turn smoke, audit, ping, TLS, flow, route sweep, API, realtime, or vitals into a 5 min–6 h loop.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/schedules`
- UI: Mission Control → **Operations** → **Schedules** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Schedules**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Test health

### Purpose

Worst-offender ranking by fail count, fail %, and flaky badge from the per-test index.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/test-health`
- UI: Mission Control → **Operations** panel → **Test health** (side rail, or ⌘K / Ctrl-K **Search**)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Test health**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Videos

### Purpose

Browse and download recorded journey videos / traces from recent jobs.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/videos`
- UI: Mission Control → **Operations** → **Videos** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Videos**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Command palette

### Purpose

⌘K / Ctrl-K or header **Search** — spotlight to jump to any action, panel, or Ask Zyra.

### When to use it

- You know the action name but not which category panel it lives in
- Fast keyboard-driven navigation without scrolling the side rail

### How to get there

- Surface: `/dashboard/command-palette`
- UI: press **⌘K** (Ctrl-K) anywhere in Mission Control, or click header **Search**

### Operate from the console (UX)

1. Open the palette from the keyboard or **Search** button.
2. Type to filter — e.g. “flow”, “Ask Zyra”, “TLS”.
3. Enter to run or navigate; Esc to close.

### Related pages

- Using the Dashboard
- Mission Control
- Page index

# Hero status

### Purpose

Cluster/app health banner with pods, replicas, last QA run, pass rate, next schedule, and knowledge (Ask Zyra) stat tile.

### When to use it

- First screen after sign-in — one glance at overall health
- Stat sub-lines explain offline cluster, empty run history, or knowledge install state

### How to get there

- Surface: `/dashboard/hero` · hash `#overview`
- UI: Mission Control → **Overview** panel (default landing)

### Operate from the console (UX)

1. Read the status lamp and headline (ALL SYSTEMS GO / DEGRADED / SYSTEMS DOWN / CLUSTER OFFLINE).
2. Scan stat tiles — pods/replicas show **No cluster connected** when kube is unavailable (not an error on a laptop).
3. **Knowledge** tile reflects Ask Zyra status; hover for install hints when offline.

Auto-refreshes every 5 s; press `r` to refresh now.

### Related pages

- Using the Dashboard
- Mission Control
- Ask Zyra
- Page index

# Live job panel

### Purpose

Live panel for the running job — macOS Terminal chrome, syntax-colored streaming log, per-test chips, Stop / download.

### When to get there

- Appears automatically when any action card starts a job (one job at a time)
- Usually visible on **Overview** or the panel where you launched the job

### How to get there

- Surface: `/dashboard/job-live`
- UI: start any action from a category panel or the ⌘K palette

### Operate from the console (UX)

1. Watch the Terminal-style log and ✓/✕ case chips while the job runs.
2. Use **Stop** to cancel mid-flight.
3. Copy or download the log; when finished, use report / video / trace links in the result area.

### Related pages

- Using the Dashboard
- Mission Control
- Page index

# Pods

### Purpose

Pod cards with phase, restarts, and click-through logs; optional cluster events toggle.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/pods`
- UI: Mission Control → **Overview** → **Pods** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Pods**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Workloads

### Purpose

Deployment and CronJob strip for the argus namespace (when kube access is available).

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/workloads`
- UI: Mission Control → **Overview** → **Workloads** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Workloads**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Mission Control

### Purpose

Live Mission Control console — grouped side rail (Console / Testing / Security / Operations), dark theme, status hero, workloads, pods, category action panels, requirements, schedules, findings, and QA run history.

### When to use it

- Your starting point for every QA action and cluster health view
- Prefer the **side rail** or **⌘K / Search** if you are unsure where to start
- Confirm `ZYVOR_BASE_URL`, dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard`
- UI: sign in at `/login` when auth is enabled, then use the **side rail** or `#overview` hash URL

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. **Overview** — hero stat tiles, workloads, pods, live job terminal.
3. **Testing panels** — Pipeline, Visual, Quality, Journeys, API, Probes, Requirements — run jobs one at a time with a live Terminal-style log.
4. **Ask Zyra** — optional citation-first knowledge Q&A (Tutorial 14).
5. **Security testing** — engagements, recon, SCA, port/TLS, DAST (injection/CSRF/SSRF/auth/IDOR), DB assert, chaos, sandboxed exploit tiers.
6. **Runs & schedules** — schedules, findings, run history, videos, test health.
7. **⌘K** or header **Search** — command palette to jump anywhere.

Use the **theme toggle** (moon/sun) for light mode; **Collapse** on the rail for icons-only.

If the console stays idle or errors, hit `GET /health`, confirm `argus serve` (or systemd `argus.service`) is up, and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Ask Zyra
- Page index

# Login

## Purpose

Mission Control sign-in when `DASHBOARD_PASSWORD` is set. Uses the same dark/light design system as the dashboard (charcoal default, blue accent buttons). Argus Enterprise uses Keycloak/OIDC with demo accounts `demo` / `demo` and `ssouser` / `Sso@321` (see the Enterprise SSO chapter in this manual).

For **Argus Enterprise** (Watchfloor) full setup steps, open Enterprise SSO / OIDC — that product does not use `DASHBOARD_PASSWORD`.

## When to use it

- You opened `/dashboard` and were redirected to `/login`
- You need to confirm Mission Control auth before running cards that read cluster logs or artifacts

## How to get there

- Surface: `/login`
- UI: open `/dashboard` when auth is enabled (redirect), or go directly to `/login`

## Operate from the console (UX)

### Mission Control (community)

1. Ensure `DASHBOARD_PASSWORD` (and optional `DASHBOARD_USER`, default `admin`) is set.
2. Open `/login`, enter username + password (lab defaults are often `admin` / `Admin@321` — rotate for anything exposed).
3. Optional: check **Remember me** for a longer session (30 days vs 12 h).
4. After success you land on `/dashboard`. `/health` and `/webhook/github` stay open.

Theme preference (`zyvor-theme` in browser storage) is shared with the dashboard after sign-in.

### Argus Enterprise (Watchfloor)

1. Claim the fresh install with the one-time setup token from app logs.
2. On the login screen, choose the SSO / Keycloak provider (or the local email/password form if local auth is enabled).
3. Demo accounts after a default Keycloak install:
   - `demo` / `demo`
   - `ssouser` / `Sso@321`
4. Verify with `scripts/test-login.sh https://argus.example.com demo demo`.

Full steps (Helm, BYO OIDC, local auth, troubleshooting): Enterprise SSO / OIDC.

If Mission Control login fails, hit `GET /health`, confirm `argus serve` is up, and re-check env from .env.example.

## Related pages

- Getting Started
- Admin basics
- Enterprise SSO / OIDC
- Using the Dashboard

- Mission Control
- Page index

---

# Create from English

### Purpose

Turn an English description into a one-off test (LLM when keyed, heuristic otherwise).

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/create`
- UI: Mission Control → **Pipeline** → **Create from English** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Create from English**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Discover coverage

### Purpose

Scan repo code/docs for untested routes and pages, then propose coverage.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/discover`
- UI: Mission Control → **Pipeline** → **Discover coverage** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Discover coverage**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Generate tests

### Purpose

Generate Playwright tests from a product spec (local path or GitHub).

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/generate`
- UI: Mission Control → **Pipeline** → **Generate tests** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Generate tests**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Run tests

### Purpose

Run ▶ Smoke (optional grep/shard) or ▶ Full LangGraph pipeline from local or GitHub specs.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/run-tests`
- UI: Mission Control → **Pipeline** → **Run tests** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Run tests**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Load test

### Purpose

Simple concurrency load test for latency under pressure.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/loadtest`
- UI: Mission Control → **Probes** → **Load test** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Load test**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Uptime ping

### Purpose

HTTP uptime / latency checks across a list of URLs.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/ping`
- UI: Mission Control → **Probes** → **Uptime ping** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Uptime ping**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Probes

### Purpose

One-shot network & security probes: redirects, headers, cookies, robots, exposed paths, API, sitemap, DNS, CORS, compression.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/probes`
- UI: Mission Control → **Probes** → **Probes** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Probes**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# TLS check

### Purpose

Certificate and DNS health for a hostname.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/tls`
- UI: Mission Control → **Probes** → **TLS check** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **TLS check**, then start the action and watch the live job panel (✓/× chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Site audit

### Purpose

Crawl pages for a11y (axe), broken links, SEO, console, perf, and security headers.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/audit`
- UI: Mission Control → **Quality** → **Site audit** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Site audit**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Crawl & test

### Purpose

Crawl an arbitrary site and validate every discovered page.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/crawl`
- UI: Mission Control → **Quality** → **Crawl & test** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Crawl & test**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Flaky check

### Purpose

Re-run the suite N times and surface unstable tests.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/flaky`
- UI: Mission Control → **Quality** → **Flaky check** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Flaky check**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Web Vitals

### Purpose

Measure Core Web Vitals (LCP / CLS / INP) with optional device and network throttle.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/vitals`
- UI: Mission Control → **Quality** → **Web Vitals** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Web Vitals**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Requirements impact

### Purpose

Impact view for requirements the pipeline ingested: shared data models and business flows, undirected co-occurrence edges (models that appear together), and typed dependencies such as **Order → Payment** — with a small SVG canvas.

### When to use it

- You want to see which requirements share an entity (e.g. `Order`)
- You care about explicit "A depends on B" relationships named in specs
- You're reviewing what a change to a model or flow might touch

### How to get there

- Surface: `/dashboard/requirements/impact` · hash `#requirements`
- UI: Mission Control → **Requirements** panel → **Impact — shared data models & flows**
- API: `GET /api/v2/requirements/impact-graph` (returns `data_models`, `flows`, `model_edges`, `model_dependencies`)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Open **Requirements** and scroll to **Impact**.
3. Browse **By data model**, **Model edges** (co-occurrence), **Typed dependencies** (directed edges + canvas), and **By flow**.
4. Click a requirement id to open its detail drawer (quality score, issues, version history, linked tests).

Entities are extracted during `evaluate_quality` (`agents/requirement_entities/`, LLM or rule fallback). Typed edges come from language like "Order depends on Payment" or the LLM's `model_dependencies` output (schema v6).

### Related pages

- Requirements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Requirements

### Purpose

Versioned requirements the pipeline ingested — source ( `github` , `document` , `email` , `transcript` , `jira` , `diarize` , …), quality score, named issues, linked tests, data models / flows, and version history (read-only).

### When to use it

- Review what the last pipeline run stored and how quality scored each item
- Open history when a requirement changed and linked tests may need review
- Jump into **Impact** for shared models, co-occurrence edges, and typed deps

### How to get there

- Surface: `/dashboard/requirements` · hash `#requirements`
- UI: Mission Control → **Requirements** panel (side rail, or ⌘K / Ctrl-K **Search**)
- APIs: `GET /api/v2/requirements` , `…/{id}` , `…/{id}/history`

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Browse the requirements list (title, source, version, quality score).
3. Click a row for description, quality issues, version history, and linked tests.
4. Use **Impact** on the same panel for shared models / flows / typed deps.

Requirements are written by the pipeline ( `fetch → parse → evaluate_quality` ), not through this UI. Sources include GitHub, local documents/PDFs, `.eml` or IMAP email, transcripts, Jira (JSON / REST / OAuth), and diarized meetings.

### Related pages

- Requirements impact
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Attack chain

### Purpose

Chains exploit-PoC steps via an LLM planner to confirm a multi-step escalation path — same sandbox and opt-ins as Exploit PoC.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/attack-chain`
- UI: Mission Control → **Security** → **Attack chain** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Attack chain**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Auth attack scan

### Purpose

Auth hygiene: JWT alg=none, cookie flags, login enum hints (no brute force) — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/auth-attack-scan`
- UI: Mission Control → **Security** → **Auth attack scan** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard auth-attack-scan`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **Auth attack scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health`, confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Chaos inject

### Purpose

Client-side fault injection (latency / loss / reset / dependency timeout) while a flow or smoke control test observes — needs ZYVOR_CHAOS_INJECTION_ENABLED, exploit engagement, and target consent.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/chaos-inject`
- UI: Mission Control → **Security** panel → **Chaos inject** (side rail, or ⌘K / Ctrl-K **Search**)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Chaos inject**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Chaos webhook

### Purpose

Trigger a customer-owned chaos experiment webhook, then run a control test with the same resilience rubric and gates as Chaos inject.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/chaos-webhook`
- UI: Mission Control → **Security** panel → **Chaos webhook** (side rail, or ⌘K / Ctrl-K **Search**)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Chaos webhook**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Cloud pentest

### Purpose

Credentialed AWS/GCP/Azure CLI enumeration of a described finding in the sandbox — same opt-ins and credential-reference rules as Host pentest.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/cloud-pentest`
- UI: Mission Control → **Security** → **Cloud pentest** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Cloud pentest**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# CSRF probe

### Purpose

Target CSRF posture: forms without tokens, cookies without SameSite — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/csrf-probe`
- UI: Mission Control → **Security** → **CSRF probe** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard csrf-probe`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **CSRF probe**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# CVE lookup

### Purpose

Read-only: fingerprints tech/versions and checks them against OSV.dev. No PoC is generated or run — requires a security engagement.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/cve-lookup`
- UI: Mission Control → **Security** → **CVE lookup** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **CVE lookup**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# DAST scan

### Purpose

Bounded DAST aggregator (headers, injection, CSRF, open-redirect; optional nuclei) — requires an exploit-tier engagement and ZYVOR_DAST_SCAN_ENABLED.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/dast-scan`
- UI: Mission Control → **Security** → **DAST scan** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard dast-scan`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **DAST scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# DB assert

### Purpose

Read-only SELECT-only assertion (row_count / cell_equals / column_values) against Postgres, MySQL, or SQLite — needs ZYVOR_DB_TESTING_ENABLED and an engagement; DSN is an env-var reference.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/db-assert`
- UI: Mission Control → **Security** panel → **DB assert** (side rail, or ⌘K / Ctrl-K **Search**)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **DB assert**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Exploit PoC

### Purpose

LLM-generated, non-destructive verification of a described finding, executed in a locked-down Kubernetes sandbox — requires an exploit-tier engagement and an execution opt-in.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/exploit-poc`
- UI: Mission Control → **Security** → **Exploit PoC** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Exploit PoC**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Host pentest

### Purpose

Credentialed SSH enumeration of a described finding via paramiko in the sandbox — needs a third, independent credentialed-pentest opt-in; credentials are always env-var references, never raw values.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/host-pentest`
- UI: Mission Control → **Security** → **Host pentest** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Host pentest**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# IDOR scan

### Purpose

Bounded adjacent-numeric-ID comparison for possible IDOR — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/idor-scan`
- UI: Mission Control → **Security** → **IDOR scan** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard idor-scan`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **IDOR scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health`, confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Injection scan

### Purpose

Systematic SQLi / reflected-XSS / path-traversal probes — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/injection-scan`
- UI: Mission Control → **Security** → **Injection scan** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard injection-scan`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **Injection scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health`, confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# LLM red-team

### Purpose

Attacker/judge adversarial-prompt battery against Ask Zyra — prompt injection, system-prompt leak, excessive agency, jailbreak, PII/secret leak — requires a security engagement.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/llm-redteam`
- UI: Mission Control → **Security** → **LLM red-team** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **LLM red-team**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Misconfig scan

### Purpose

Tech/version fingerprinting, wordlist-driven path discovery, security-header value grading, and DNS hygiene checks — requires a security engagement.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/misconfig-scan`
- UI: Mission Control → **Security** → **Misconfig scan** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Misconfig scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Port scan

### Purpose

Bounded TCP connect scan of common service ports (≤64) — requires an active_recon engagement.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/port-scan`
- UI: Mission Control → **Security** → **Port scan** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard port-scan`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **Port scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# SCA scan

### Purpose

Client-side library/license fingerprinting and/or local-checkout pip-audit/npm audit — URL mode needs an engagement; checkout mode is operator-local.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/sca-scan`
- UI: Mission Control → **Security** panel → **SCA scan** (side rail, or ⌘K / Ctrl-K **Search**)

### What you can do

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **SCA scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Security engagements

### Purpose

Create/revoke the admin-issued, target-scoped authorization required before running misconfig scan, CVE lookup, SCA, DAST / network-attack jobs, LLM red-team, or sandboxed exploit tiers.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/security-engagements`
- UI: Mission Control → **Security** → **Security engagements** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Security engagements**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# SSRF probe

### Purpose

Probes URL-like query params for server-side fetch / SSRF signals — requires exploit engagement and ZYVOR_DAST_SCAN_ENABLED.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/ssrf-probe`
- UI: Mission Control → **Security** → **SSRF probe** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard ssrf-probe`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **SSRF probe**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# TLS cipher scan

### Purpose

TLS protocol and weak-cipher grading beyond basic cert metadata — requires an active_recon engagement.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm a live security engagement and any required opt-in flags (see purpose) before starting

### How to get there

- Surface: `/dashboard/actions/tls-cipher-scan`
- UI: Mission Control → **Security** → **TLS cipher scan** (side rail panel, or ⌘K / Ctrl-K Search)
- CLI: `argus guard tls-cipher-scan`

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Create or select an authorized **Security engagement**, fill the card fields for **TLS cipher scan**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any report links the card produces.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example. See also Network-attack / DAST gaps.

### Related pages

- Security engagements
- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Compare URLs

### Purpose

Side-by-side visual diff of two URLs (e.g. staging vs production).

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/compare`
- UI: Mission Control → **Visual** → **Compare URLs** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Compare URLs**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Visual regression

### Purpose

Pixel-compare visual snapshots against baselines; optionally update baselines.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/regression`
- UI: Mission Control → **Visual** → **Visual regression** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Visual regression**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Route sweep

### Purpose

Screenshot many routes (or auto-crawl) and diff against route-sweep baselines.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/route-sweep`
- UI: Mission Control → **Visual** → **Route sweep** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Route sweep**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Screenshot

### Purpose

Capture desktop / tablet / mobile screenshots of any URL.

### When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** ( `/dashboard` ) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

### How to get there

- Surface: `/dashboard/actions/screenshot`
- UI: Mission Control → **Visual** → **Screenshot** (side rail panel, or ⌘K / Ctrl-K Search)

### Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Screenshot**, then start the action and watch the live job panel (✓/✕ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up ( `argus serve` ), and re-check env from .env.example.

### Related pages

- Getting Started
- Using the Dashboard
- Mission Control
- Page index

# Zyvor Argus — Complete page index

Every Mission Control surface and action card.

*52 routes*

Regenerate: `node scripts/customer-docs/generate-page-index.mjs`

## Overview

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Mission Control | `/dashboard` | Live Mission Control console — grouped side rail (Console / Testing / Security / Operations), dark theme, status hero, workloads, pods, category action panels, requirements, schedules, findings, and QA run history. | Open |
| Login | `/login` | Mission Control sign-in — same dark/light design system as the dashboard; DASHBOARD_PASSWORD gate. Argus Enterprise uses Keycloak SSO (demo/demo) — see customer/enterprise-sso.md. | Open |
| Hero status | `/dashboard/hero` | Cluster/app health banner with pods, replicas, last QA run, pass rate, next scheduled smoke, and knowledge (Ask Zyra) status. | Open |
| Workloads | `/dashboard/workloads` | Deployment and CronJob strip for the argus namespace (when kube access is available). | Open |
| Pods | `/dashboard/pods` | Pod cards with phase, restarts, and click-through logs; optional cluster events toggle. | Open |
| Live job panel | `/dashboard/job-live` | Live panel for the running job — macOS Terminal chrome, syntax-colored log, per-test chips, Copy / Save / Stop. | Open |
| Command palette | `/dashboard/command-palette` | ⌘K / Ctrl-K or header Search — spotlight to jump to any action, panel, or Ask Zyra. | Open |

## Console

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Ask Zyra | `/dashboard/ask` | Ask Zyra — citation-first product knowledge Q&A (optional Qdrant RAG); read-only, no cluster mutations. | Open |

## Pipeline

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Run tests | `/dashboard/actions/run-tests` | Run ▶ Smoke (optional grep/shard) or ▶ Full LangGraph pipeline from local or GitHub specs. | Open |
| Generate tests | `/dashboard/actions/generate` | Generate Playwright tests from a product spec (local path or GitHub). | Open |
| Discover coverage | `/dashboard/actions/discover` | Scan repo code/docs for untested routes and pages, then propose coverage. | Open |
| Create from English | `/dashboard/actions/create` | Turn an English description into a one-off test (LLM when keyed, heuristic otherwise). | Open |

## Visual

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Visual regression | `/dashboard/actions/regression` | Pixel-compare visual snapshots against baselines; optionally update baselines. | Open |
| Compare URLs | `/dashboard/actions/compare` | Side-by-side visual diff of two URLs (e.g. staging vs production). | Open |
| Screenshot | `/dashboard/actions/screenshot` | Capture desktop / tablet / mobile screenshots of any URL. | Open |
| Route sweep | `/dashboard/actions/route-sweep` | Screenshot many routes (or auto-crawl) and diff against route-sweep baselines. | Open |

## Quality

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Site audit | `/dashboard/actions/audit` | Crawl pages for a11y (axe), broken links, SEO, console, perf, and security headers. | Open |
| Flaky check | `/dashboard/actions/flaky` | Re-run the suite N times and surface unstable tests. | Open |
| Web Vitals | `/dashboard/actions/vitals` | Measure Core Web Vitals (LCP / CLS / INP) with optional device and network throttle. | Open |
| Crawl & test | `/dashboard/actions/crawl` | Crawl an arbitrary site and validate every discovered page. | Open |

## Probes

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Uptime ping | `/dashboard/actions/ping` | HTTP uptime / latency checks across a list of URLs. | Open |
| Load test | `/dashboard/actions/loadtest` | Simple concurrency load test for latency under pressure. | Open |
| TLS check | `/dashboard/actions/tls` | Certificate and DNS health for a hostname. | Open |
| Probes | `/dashboard/actions/probes` | One-shot network & security probes: redirects, headers, cookies, robots, exposed paths, API, sitemap, DNS, CORS, compression. | Open |

## Journeys

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Flow test | `/dashboard/actions/flow` | Multi-step user journey (English or step DSL) with optional login/session, video, and trace. | Open |
| HAR record / replay | `/dashboard/actions/har` | Record a HAR while browsing routes, or replay the UI against a captured HAR. | Open |

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Import codegen | `/dashboard/actions/import-codegen` | Paste Playwright codegen JS/TS and convert it into flow steps (optionally run). | Open |
| AI test | `/dashboard/actions/ai-test` | Describe a goal; the autonomous browser agent drives the app toward it. | Open |

### API

| Page | Route | Purpose | Guide |
|---|---|---|---|
| API contract | `/dashboard/actions/api-contract` | Validate REST endpoints against an OpenAPI schema (Forge preset available). | Open |
| API contract diff | `/dashboard/actions/api-contract-diff` | Static OpenAPI breaking-change diff between two specs (URL, git::, or inline JSON) — no live target, no engagement gate. | Open |
| Contract verify | `/dashboard/actions/contract-verify` | HAR-derived consumer contract verification against a live provider (status, content-type, top-level JSON keys) — requires an active_recon engagement. | Open |
| Live data | `/dashboard/actions/realtime` | Assert WebSocket / SSE streams and optional live-region updates. | Open |
| Auth & session | `/dashboard/actions/auth` | Login → reusable session file → logout / expiry / negative auth checks. | Open |

### Requirements

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Requirements | `/dashboard/requirements` | Versioned requirements the pipeline ingested — source, quality score, named issues, linked tests, and version history (read-only). | Open |
| Requirements impact | `/dashboard/requirements/impact` | Impact view — shared data models & flows, co-occurrence edges, and typed Order → Payment dependencies with SVG canvas. | Open |

### Security

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Security engagements | `/dashboard/actions/security-engagements` | Create/revoke the admin-issued, target-scoped authorization required before running any deeper security testing job. | Open |
| Misconfig scan | `/dashboard/actions/misconfig-scan` | Tech/version fingerprinting, path discovery, header grading, DNS hygiene, plus compliance signals (security.txt, consent markers, PII patterns) — requires a security engagement. | Open |
| CVE lookup | `/dashboard/actions/cve-lookup` | Read-only: fingerprints tech/versions and checks them against OSV.dev. No PoC is generated or run — requires a security engagement. | Open |

| Page | Route | Purpose | Guide |
|---|---|---|---|
| SCA scan | `/dashboard/actions/sca-scan` | Client-side library/license fingerprinting and/or local-checkout pip-audit/npm audit — URL mode needs an engagement; checkout mode is operator-local. | Open |
| LLM red-team | `/dashboard/actions/llm-redteam` | Attacker/judge adversarial-prompt battery against Ask Zyra — prompt injection, system-prompt leak, excessive agency, jailbreak, PII/secret leak — requires a security engagement. | Open |
| Exploit PoC | `/dashboard/actions/exploit-poc` | LLM-generated, non-destructive verification of a described finding, executed in a locked-down Kubernetes sandbox — requires an exploit-tier engagement and an execution opt-in. | Open |
| Attack chain | `/dashboard/actions/attack-chain` | Chains exploit-PoC steps via an LLM planner to confirm a multi-step escalation path — same sandbox and opt-ins as Exploit PoC. | Open |
| Host pentest | `/dashboard/actions/host-pentest` | Credentialed SSH enumeration of a described finding via paramiko in the sandbox — needs a third, independent credentialed-pentest opt-in; credentials are always env-var references, never raw values. | Open |
| Cloud pentest | `/dashboard/actions/cloud-pentest` | Credentialed AWS/GCP/Azure CLI enumeration of a described finding in the sandbox — same opt-ins and credential-reference rules as Host pentest. | Open |
| DB assert | `/dashboard/actions/db-assert` | Read-only SELECT-only assertion (row_count / cell_equals / column_values) against Postgres, MySQL, or SQLite — needs ZYVOR_DB_TESTING_ENABLED and an engagement; DSN is an env-var reference. | Open |
| Chaos inject | `/dashboard/actions/chaos-inject` | Client-side fault injection (latency / loss / reset / dependency timeout) while a flow or smoke control test observes — needs ZYVOR_CHAOS_INJECTION_ENABLED, exploit engagement, and target consent. | Open |
| Chaos webhook | `/dashboard/actions/chaos-webhook` | Trigger a customer-owned chaos experiment webhook, then run a control test with the same resilience rubric and gates as Chaos inject. | Open |

## Operations

| Page | Route | Purpose | Guide |
|---|---|---|---|
| Schedules | `/dashboard/schedules` | Turn smoke, audit, ping, TLS, flow, route sweep, API, realtime, or vitals into a 5 min–6 h loop (Runs & schedules panel). | Open |
| Findings | `/dashboard/findings` | Collected issues from API, auth, live-data, vitals, audit, and security jobs with export/clear. | Open |
| QA Runs | `/dashboard/runs` | History table of QA runs with pass/fail chips and sparkline trends. | Open |
| Videos | `/dashboard/videos` | Browse and download recorded journey videos / traces from recent jobs. | Open |

| Page | Route | Purpose | Guide |
| --- | --- | --- | --- |
| Test health | `/dashboard/test-health` | Worst-offender ranking by fail count, fail %, and flaky badge from the per-test index. | Open |

## Related

- Customer docs home
- Page-by-page guides